



DuocUC[®] INFORMÁTICA Y
TELECOMUNICACIONES



API Gateway + Service Discovery con Eureka

DESARROLLO FULLSTACK I
DSY1103



• Contenidos

- Introducción a YAML
- Rol del API Gateway en Microservicios
- Eureka
- Desarrollo de guía práctica.

A black and white photograph of a man in a suit holding a tablet, standing in a modern office with glass partitions and a long desk. The scene is dimly lit, with a bright light source visible in the background. The man is smiling and looking at the tablet. The office environment includes a desk with a coffee cup and some papers, and a chair in the foreground.

01

YAML

• Introducción a YAML

YAML (YAML Ain't Markup Language) es un formato de serialización de datos diseñado para ser fácilmente legible por humanos. Es la elección preferida en muchos proyectos de microservicios (especialmente Kubernetes y Spring Cloud) por su claridad al manejar estructuras complejas. Su sintaxis se basa en sangrías (similar a Python), pares clave-valor y no utiliza llaves ni corchetes, lo que facilita su lectura para las personas.

Características clave

- **Legibilidad:** Está diseñado para ser fácil de leer y escribir por humanos, con una estructura limpia.
- **Serialización de datos:** Permite representar estructuras de datos complejas en un formato simple y estandarizado.
- **Sintaxis:** Utiliza sangría para definir la estructura, similar a Python, en lugar de etiquetas o corchetes.
- **Comentarios:** Se pueden añadir comentarios con el símbolo de almohadilla (#).
- **Extensiones de archivo:** Los archivos suelen tener la extensión .yaml o .yml.



YAML

• Sintaxis Básica de YAML

La sintaxis de YAML se basa en tres estructuras principales:

A. Mapeos (Objetos / Diccionarios): Se utilizan para representar estructuras jerárquicas (llave-valor).

La indentación con espacios (nunca tabulaciones) define la anidación.

Properties

```
server.port=8080
spring.application.name=g
ateway
```

YAML

```
server:
  port: 8080

spring:
  application:
    name: gateway
```

• Sintaxis Básica de YAML

B. Secuencias (Arrays / Listas): Se utilizan para representar listas ordenadas de elementos. Se indican con un guion (-) seguido de un espacio.

```
Properties      YAML
my.list[0]=ite  my:
m1
my.list[1]=ite  list:
m2              - item1
                 - item2
```

C. Tipos de Datos: YAML maneja automáticamente tipos de datos básicos (cadenas, números, booleanos). Las cadenas a menudo no necesitan comillas.

```
Tipo           YAML
String          name: My
(Cadena)         Service
Number          port: 8080
(Número)
Boolean         enabled: true
(Booleano)
```

• Ventajas de YAML en Spring Cloud Gateway

En el contexto de la configuración distribuida y el API Gateway, YAML ofrece grandes beneficios:

Claridad de Rutas: La definición de rutas, predicados y filtros en Spring Cloud Gateway es una estructura de lista anidada. YAML maneja esto de forma natural, haciendo que el archivo de configuración sea mucho más fácil de leer y mantener que el formato properties.

```
# YAML es visualmente superior para esta estructura
spring:
  cloud:
    gateway:
      routes: # Esto es una Secuencia/Lista
        - id: orders_route # Esto es un Mapeo (el primer elemento de la lista)
          uri: lb://ORDERS-SERVICE
          predicates: # Esto es una Secuencia/Lista
            - Path=/api/orders/**
```


• Ventajas de YAML en Spring Cloud Gateway

Perfiles Simplificados: Al definir configuraciones específicas para entornos (dev, prod, test), YAML permite usar el símbolo `---` para separar las configuraciones de los perfiles en un único archivo `application.yml`.

```
# application.yml
server:
  port: 8080 # Configuración base

---

spring:
  config:
    activate:
      on-profile: prod # Configuración para Perfil 'prod'
server:
  port: 443 # Puerto distinto en Producción
```


• Ventajas de YAML en Spring Cloud Gateway

Menos Duplicación: Al usar la jerarquía, solo tienes que escribir la clave padre una vez. En properties, tendrías que repetir `spring.cloud.gateway.routes` para cada línea de configuración.

Consideraciones

Indentación es Clave: Un solo espacio extra o una tabulación en lugar de espacios arruinará el archivo YAML. ¡Sé estricto con la indentación!



02

API Gateway

• ¿Qué es un API Gateway?

Un **API Gateway** es un componente central de una arquitectura de microservicios que actúa como:

- **Punto único de entrada** para todos los clientes (web, móvil, Postman).
- **Orquestador de rutas** hacia los microservicios internos.
- **Capa de seguridad**, donde se pueden aplicar validaciones, autenticación o control de tráfico.
- **Capa de filtros transversales**, como logs, manejo de cabeceras, rate-limiting o CORS.

En lugar de exponer múltiples microservicios a los clientes, el Gateway concentra todo en una única URL.

El cliente accede a:

```
GET http://localhost:8080/api/orders
```

El Gateway dirige esa petición internamente a:

```
http://localhost:8082/orders
```

[Spring Cloud Gateway](#)

• ¿Qué es un API Gateway?

Características y Conceptos Clave

Concepto	Descripción
Enrutamiento (Routing)	La función principal. Inspecciona la petición entrante (por path, header, método HTTP) y la redirige al microservicio interno correcto.
Predicados (Predicates)	Son las condiciones que el Gateway evalúa para decidir si una petición coincide con una ruta. Ejemplos: Path=/users/**, Method=POST.
Filtros (Filters)	Permiten modificar la petición o la respuesta. Se ejecutan antes (pre-filter) o después (post-filter) del enrutamiento.
Servicios Transversales	Funcionalidades aplicadas a múltiples servicios desde un solo lugar: Seguridad (JWT, OAuth2), Rate Limiting (límite de peticiones), Logging/Monitoring .

• ¿Qué es un API Gateway?

Ventajas de Usar un Gateway

Ventaja	Explicación
Seguridad Centralizada	Autenticar y autorizar al cliente una sola vez en el Gateway, simplificando la lógica en los microservicios internos.
Reducción de Latencia	Optimiza el número de peticiones. Un cliente puede hacer una sola llamada al Gateway en lugar de múltiples llamadas a diferentes microservicios.
Gestión de Versiones	Permite exponer diferentes versiones de API (/v1/users vs /v2/users) sin modificar los servicios internos.
Desacoplamiento	Permite que los servicios internos cambien de ubicación o tecnología sin afectar a los clientes.



• Construcción Básica del API Gateway

¿Cómo Decide el Gateway las URLs?

El Gateway es un microservicio aparte que necesita saber dos cosas:

URL de Entrada (Puerta Pública)

La URL de entrada es la que expones públicamente. Esto se configura con el puerto del Gateway

Mecanismo de Llamada a Microservicios (Rutas)

El Gateway utiliza la configuración de Rutas para mapear la URL de entrada a la URL interna del microservicio.

Método Inicial: Rutas Fijas (Sin Eureka)

Al inicio, se pueden usar URLs y puertos fijos. Esto no es escalable, pero ilustra el concepto



03

Eureka

• ¿Qué es un Service Discovery?

El **Service Discovery (Descubrimiento de Servicios)** es el proceso por el cual una aplicación o servicio encuentra la ubicación de red (IP y puerto) de otro servicio con el que necesita comunicarse.

En una arquitectura de microservicios, las instancias de los servicios son efímeras: se crean, se eliminan, se escalan y se mueven dinámicamente. Esto hace inviable usar URLs fijas y preconfiguradas.

El Service Discovery permite que los servicios se registren en un servidor central y otros servicios (como el Gateway) puedan descubrirlos automáticamente sin codificar las URLs

• ¿Qué es Eureka?

Eureka es un **Service Registry y Service Discovery** desarrollado por Netflix y adoptado por Spring Cloud.

Su función es permitir que los microservicios se descubran entre sí dinámicamente, sin necesidad de codificar URLs o puertos en el código fuente.

Eureka implementa el patrón “**Client-Side Service Discovery**”, donde:

- Cada microservicio se registra en Eureka.
- Eureka mantiene una lista de servicios disponibles.
- Los clientes (como Gateway, webapps o servicios internos) consultan a Eureka para resolver el service-id en tiempo real.
- El cliente decide a qué instancia conectarse, permitiendo balancear carga de forma nativa.

A green rounded rectangular button with the word "EUREKA" in blue, underlined capital letters.

• Service Discovery con Eureka

Eureka (Spring Cloud Netflix Eureka) es la implementación estándar del patrón Service Discovery. Resuelve el problema de las rutas fijas.

En una arquitectura distribuida, los microservicios:

- Pueden iniciarse en puertos aleatorios.
- Pueden escalarse (tener múltiples instancias en diferentes IPs).
- Pueden fallar y ser reemplazados.

Eureka proporciona:

- **Service Registry (Servidor):** Un servidor central donde todos los microservicios se registran al iniciarse.
- **Service Client (Cliente):** La dependencia que cada microservicio y el Gateway usan para registrarse o consultar la ubicación de otros.



• ¿Qué Cambia la Integración de Eureka?

La integración con Eureka transforma las rutas del API Gateway de estáticas a dinámicas

Sin Eureka (Rutas Fijas)

uri: http://localhost:8081

El Gateway apunta a una ubicación de red específica.

No hay tolerancia a fallos ni balanceo de carga.

El cliente necesita reiniciar si la IP del servicio cambia.

Con Eureka (Rutas Dinámicas)

uri: lb://ORDERS-SERVICE

El Gateway apunta a un nombre lógico (ORDERS-SERVICE).

Eureka proporciona las IPs. **lb://** activa el Client-Side Load Balancing y la tolerancia a fallos.

La ubicación puede cambiar dinámicamente sin reiniciar el Gateway.

• ACTIVIDAD PRÁCTICA



Investiga la implementación de **Eureka** en tus proyectos e intenta cambiar uno de los desarrollados en clases a este nuevo sistema.

DuocUC[®]

CERCANÍA. LIDERAZGO. FUTURO.

duoc.cl

7 AÑOS
ACREDITADO



DESDE AGOSTO 2017 HASTA AGOSTO 2024.
DOCENCIA DE PREGRADO. GESTIÓN
INSTITUCIONAL. VINCULACIÓN CON EL MEDIO.